SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Sree B	**Reg. No.:**	192524023
Section / Batch:	AI and DS / 2025	**Date:**	01/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Debugging Closest Pair Code (Minimum Distance Updated with Pair Index Difference)
The following brute force Closest Pair implementation calculates Euclidean distance correctly, but it updates the minimum value using the index difference instead of the actual distance in one branch of the logic. Carefully analyze why this produces meaningless results for many point configurations. Apply systematic debugging to ensure that only true geometric distance is stored and compared throughout the execution. Optimize the implementation by avoiding redundant tuple access where possible. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
import math
def closest_pair(points):
min_dist = float(&amp;#39;inf&amp;#39;)

for i in range(len(points)):
for j in range(i+1, len(points)):
dist = math.sqrt((points[i][0]-points[j][0])**2 +
(points[i][1]-points[j][1])**2)
if dist &amp;lt; min_dist:
min_dist = j - i # ERROR
return min_dist
```

Q126
Debugging Convex Hull Orientation Logic (Positive and Negative Sides Not Distinguished in C)
The following brute force Convex Hull helper in C uses the same flag update for both positive and negative orientation values, which prevents it from correctly determining whether points lie on both sides of a candidate edge. Carefully analyze why hull validation depends on tracking both sides independently. Apply systematic debugging so that positive and negative orientations are recorded separately and mixed-side edges are rejected correctly. Optimize the corrected

implementation for clear geometric reasoning and reduced unnecessary checks. Analyze the time complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```
int isHullEdge(int p1, int p2, int x[], int y[], int n) {
int i, pos = 0, neg = 0;
for (i = 0; i &lt; n; i++) {
int val = (y[p2] - y[p1]) * (x[i] - x[p2]) - (x[p2] - x[p1]) * (y[i] -
y[p2]);
if (val &gt; 0)
pos = 1;
if (val &gt; 0) // ERROR
neg = 1;
}
return !(pos &amp;&amp; neg);
```

Code of Conduct

I Sree B(192524023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sree B

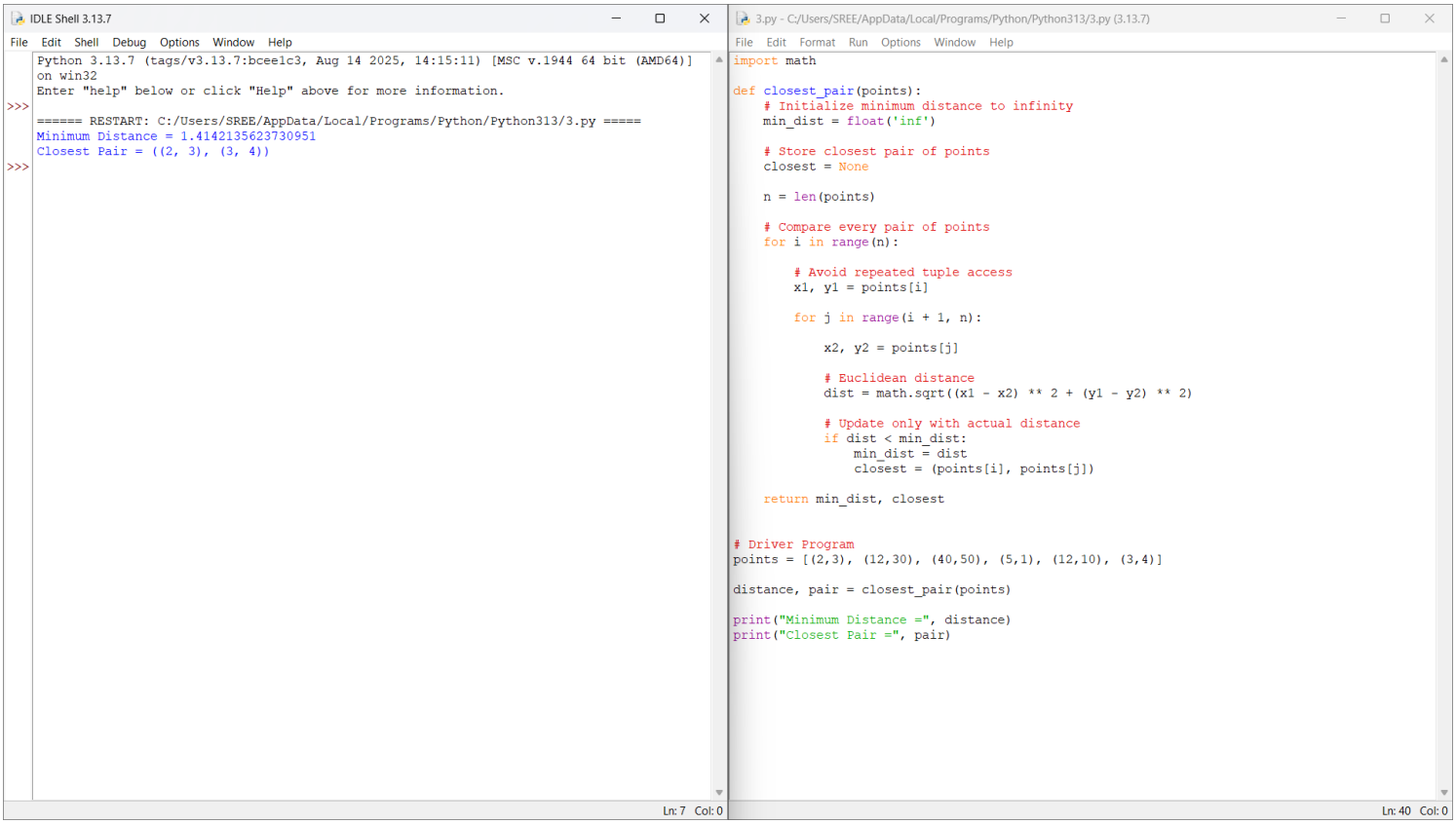
RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.py =====
Minimum Distance = 1.4142135623730951
Closest Pair = ((2, 3), (3, 4))
>>>
Ln: 7 Col: 0
3.py - C:/Users/SREE/AppData/Local/Programs/Python/Python313/3.py (3.13.7)
File Edit Format Run Options Window Help
import math
def closest_pair(points):
# Initialize minimum distance to infinity
min_dist = float('inf')
# Store closest pair of points
closest = None
n = len(points)
# Compare every pair of points
for i in range(n):
# Avoid repeated tuple access
x1, y1 = points[i]
for j in range(i + 1, n):
x2, y2 = points[j]
# Euclidean distance
dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
# Update only with actual distance
if dist < min_dist:
min_dist = dist
closest = (points[i], points[j])
return min_dist, closest
# Driver Program
points = [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)]
distance, pair = closest_pair(points)
print("Minimum Distance =", distance)
print("Closest Pair =", pair)
Ln: 40 Col: 0

```c
#include <stdio.h>

/* Returns 1 if edge (p1,p2) is part of Convex Hull */
int isHullEdge(int p1, int p2, int x[], int y[], int n)
{
    int i;
    int pos = 0;
    int neg = 0;

    for (i = 0; i < n; i++)
    {
        /* Ignore the endpoints */
        if (i == p1 || i == p2)
            continue;

        /* Orientation (cross product) */
        int val = (y[p2] - y[p1]) * (x[i] - x[p2]) -
                  (x[p2] - x[p1]) * (y[i] - y[p2]);

        if (val > 0)
            pos = 1;

        else if (val < 0)
            neg = 1;

```

```
            this cannot be a hull edge. */
        if (pos && neg)
            return 0;
    }

    return 1;
}

int main()
{
    int x[] = {0, 0, 3, 3, 1};
    int y[] = {0, 3, 3, 0, 1};

    int n = 5;

    if (isHullEdge(0, 1, x, y, n))
        printf("Edge belongs to Convex Hull\n");
    else
        printf("Edge does NOT belong to Convex Hull\n");

    return 0;
}
```

```
Edge belongs to Convex Hull
```